Peer to peer Networks / Onion Routing

Anonymous Communication in Distributed Systems

Zhivko Stoimchev
89221056@student.upr.si
UP FAMNIT

ABSTRACT

Growing concerns around online privacy and traffic analysis have made anonymous communication an increasingly relevant problem in modern networking. Standard internet traffic exposes the source and destination of every connection, allowing third parties to monitor user behaviour without ever reading the content itself. ZMIX is a peer-to-peer onion routing network implemented from scratch in Java 21 that addresses this problem directly.

The system builds a self-organizing overlay network through gossip-based peer discovery, then constructs multi-hop anonymous circuits using Elliptic Curve Diffie-Hellman key agreement. HTTP traffic is anonymized by wrapping it in nested layers of AES-256-GCM encryption — one per circuit hop — so that each relay node only decrypts its own layer and learns nothing beyond its immediate neighbours. The implementation covers a custom message protocol, concurrent peer and circuit management, a CLI and Swing GUI for issuing requests, and a Docker Compose setup for simulating a distributed network environment.

KEYWORDS

Onion Routing, Peer-to-Peer Networks, Anonymous Communication, Network Security, Public Key Cryptography, Docker, Java 21

1 INTRODUCTION

Traditional internet communication, even when encrypted, exposes metadata that reveals who is talking to whom, how often, and for how long. This information alone is enough for traffic analysis and identity disclosure, without ever breaking the encryption on the content. Onion routing solves this by routing traffic through a chain of intermediary nodes called a circuit, where each node only knows its immediate predecessor and successor. The message is wrapped in multiple layers of encryption — one per hop — and each node peels exactly one layer before forwarding the rest. No single node ever sees both the origin and the destination at the same time.

ZMIX implements this concept over a fully decentralized peer-to-peer network. Nodes discover each other through a gossip protocol, negotiate per-hop session keys using ECDH, and route HTTP requests through layered AES-256-GCM encryption. The project is built entirely in Java 21 with no external cryptography libraries, and uses Docker Compose to simulate a live multi-node environment. The rest of the report covers the system architecture in Section 2, the protocol design and circuit construction in Section 3, the security design in Section 4, data transfer in Section 5, deployment and setup in Section 6, and limitations and future work in Section 7.

2 SYSTEM ARCHITECTURE

ZMIX is a fully decentralized overlay network where every node runs the same binary. There are no dedicated roles — whether a node acts as a relay, exit node, or circuit initiator depends entirely on the network state at runtime. The only exception is the bootstrap node, which serves as the initial seed for peer discovery and does not attempt outbound connections on startup.

Internally the design is split into three layers: transport (TCP sockets and message serialization), network management (peer registry and connection maintenance), and protocol logic (peer discovery and circuit operations). These layers communicate through a shared event bus — a `BlockingQueue<Event>` — so that I/O threads never directly invoke protocol logic. Every received message is placed into the queue and processed by a single dedicated consumer thread.

2.1 Component Overview

The following components make up the system:

- **Server** – Listens for inbound TCP connections and spawns a `Peer` thread for each accepted connection.
- **Peer** – Handles the handshake, reads inbound messages, and places them onto the shared event queue.
- **NetworkManager** – Central registry for connected and known peers. Drives scheduled connection maintenance.
- **MessageHandler** – Single-threaded event queue consumer. Dispatches each message to the appropriate protocol handler.
- **PeerDiscoveryProtocol** – Handles peer list exchange and periodically broadcasts discovery requests to all connected peers.
- **CircuitManager** – Manages circuit state and handles all circuit-related messages.
- **Crypto** – Provides ECDH key agreement, SHA-256 key derivation, and AES-256-GCM encryption and decryption.
- **Config** – Loads `.properties` configuration files and applies environment variable overrides for Docker deployments.
- **Cli / Gui** – User interfaces for submitting URLs into the circuit request queue via terminal or Swing window.

2.2 Thread Model

ZMIX uses an explicit, minimal threading model. Each peer connection runs on its own thread from a `CachedThreadPool`. All protocol logic runs on the single `MessageHandler` thread, which means handlers in `CircuitManager` and `PeerDiscoveryProtocol` do not need internal synchronization — they are never called concurrently. Circuit construction runs on a separate scheduled thread, namely `SingleThreadScheduledExecutor` so it does not block peer I/O,

and once the circuit is active a dedicated `requestExecutor` processes queued URLs one at a time. Two scheduled tasks run in the background: one for periodic peer discovery broadcasts and one for connection maintenance.

3 PROTOCOL DESIGN

3.1 Message Format

All communication between nodes uses a custom text-based protocol over persistent TCP connections. Each message is a single newline-terminated string with four fields separated by a fixed delimiter:

`MessageType` **`;delim;;;;`** `Timestamp` **`;delim;;;;`** `MessageId` **`;delim;;;;`** `Payload`

The delimiter `;delim;;;;` was chosen to avoid collisions with Base64 or UUID characters that appear in the payload fields. Binary data such as public keys and encrypted bytes is Base64-encoded before embedding. Each payload type implements a `toBytes()` and `fromBytes()` contract, keeping serialization logic inside the payload class itself.

3.2 Message Types

- **HANDSHAKE** – First message on every new connection. Carries the sender's public key and listening port.
- **PEER_DISCOVERY_REQUEST** – Requests the recipient's known peer list. No payload.
- **PEER_DISCOVERY_RESPONSE** – Return a list of known peers in the format `publicKey;host;port`.
- **CIRCUIT_CREATE_REQUEST** – Initiate a key exchange. Carries a circuit ID and an ephemeral public key.
- **CIRCUIT_CREATE_RESPONSE** – Returns the target node's ephemeral public key to complete the exchange.
- **CIRCUIT_EXTEND_REQUEST** – Tunnels a circuit creation request to the next hop through the existing circuit.
- **CIRCUIT_EXTEND_RESPONSE** – Return the next hop's ephemeral key back to the initiator.
- **DATA_TRANSFER_REQUEST** – Sends an encrypted (layered) HTTP request through the circuit.
- **DATA_TRANSFER_RESPONSE** – Returns the HTTP response back along the reverse path.

3.3 Handshake & Peer Registration

Every new TCP connection starts with a handshake before any other message is processed. The outbound node sends its `HANDSHAKE` first; the inbound node waits, validates it, and then sends its own. A five-second socket timeout is set during the exchange to guard against hung connections, and reset to zero once the handshake completes.

The handshake payload carries the node's long-term EC public key and its listening port. This is important because the socket's remote port is the ephemeral OS-assigned port, not the port the node is actually listening on — so the listening port must be communicated explicitly.

After a successful handshake, `NetworkManager` is registering the peer. If the node has already reached its maximum connection limit, it sends the new peer a short list of known peers and disconnects immediately, acting as a lightweight referral rather than refusing the connection silently.

```
byte[] encrypted = payload.toBytes();
for (int i = hop - 1; i >= 0; i--) {
    encrypted = crypto.encryptAES(encrypted, keys.
        get(i));
}
```

Listing 1: Encrypting the extend payload through established hops

3.4 Peer Discovery

Peer discovery uses a simple gossip protocol seeded by the bootstrap node. When an outbound connection completes its handshake, the peer immediately sends a `PEER_DISCOVERY_REQUEST`. The receiving node replies with its full known peer list. The requesting node merges any new entries into its own list, filtering out itself and peers it already knows.

Two scheduled tasks keep the topology healthy over time. To start with, `PeerDiscoveryProtocol` periodically broadcasts a message of type `PEER_DISCOVERY_REQUEST` to all connected peers. Afterwards, `NetworkManager.startPeerMaintenance()` attempts outbound connections to known-but-disconnected peers until the minimum connection floor defined by `node.connections.min` is satisfied.

3.5 Circuit Construction

Circuit construction is the most important part of the protocol. Rather than sending all routing information in a single message, the initiator builds the circuit incrementally — negotiating a shared secret with each hop one at a time, always tunnelling through the hops already established. This ensures no relay ever learns the full path.

For a three-hop circuit through nodes E (entry), M (middle), and X (exit), the process works as follows:

Step 1 — Entry node. The initiator connects to E and sends a `CIRCUIT_CREATE_REQUEST` carrying a fresh ephemeral EC public key. E generates its own ephemeral pair, performs ECDH, and replies with its key in a `CIRCUIT_CREATE_RESPONSE`. Both sides independently derive the same AES session key K0.

Step 2 — Middle node. The initiator wraps a payload of type `CircuitExtendRequestPayload` containing M's address and a new ephemeral key for M, encrypts it with K0, and sends it as a message of type `CIRCUIT_EXTEND_REQUEST` to E. E decrypts, connects to M, and forwards a `CIRCUIT_CREATE_REQUEST` on behalf of the initiator. M's ephemeral key travels back through E re-encrypted with K0, and the initiator derives K1.

Step 3 — Exit node. The same process repeats for X, but the extend payload is now encrypted first with K1 and then with K0. E peels K0, M peels K1, and X receives the plaintext create request. X's response travels back re-encrypted at each relay, and the initiator derives K2.

Once all three exchanges complete, the circuit status is set to `ACTIVE`. The initiator holds `[K0, K1, K2]`, while each relay only knows its own key and its immediate neighbours.

4 SECURITY DESIGN

4.1 Key Agreement

Each hop in the circuit gets its own independent session key, negotiated using Elliptic Curve Diffie-Hellman on the secp256r1 (P-256) curve. For every hop, both the initiator and the target node generate a fresh ephemeral key pair. They exchange public keys through the circuit messages, perform ECDH locally, and arrive at the same shared secret without ever transmitting it over the network.

The key exchange happens entirely through the circuit messages described in Section 3. The long-term identity key is never used for encryption — it only identifies the node during the handshake and peer discovery.

4.2 Key Derivation

The raw ECDH output is not used directly as an AES key. SHA-256 is applied to the shared secret and the first 32 bytes of the result are used as the AES-256 key. This ensures the key material is the correct length and has uniform distribution before being passed to the cipher.

4.3 Symmetric Encryption

All data encryption uses AES-256-GCM. GCM is an authenticated encryption mode, providing both confidentiality and integrity in a single operation. Every encryption call generates a fresh random 12-byte IV using `SecureRandom`, which is prepended to the ciphertext before sending. Decryption extracts the IV from the first 12 bytes and verifies the GCM authentication tag before returning the plaintext. Any tampering with the ciphertext causes decryption to fail rather than silently return corrupted data.

This matters in the context of onion routing because each relay is blindly forwarding payloads it cannot read. GCM authentication ensures a malicious relay cannot modify data passing through it without detection.

4.4 Forward Secrecy

Because each circuit uses freshly generated ephemeral key pairs that are discarded immediately after the session key is derived, compromising a node's long-term private key at a later point does not reveal past session keys. An attacker who records all circuit traffic and later breaks into a relay gains nothing — the ephemeral keys that produced those session keys no longer exist.

5 DATA TRANSFER

5.1 Sending a Request

Once the circuit is `ACTIVE`, the user submits a URL through the CLI or GUI. The URL is parsed to extract the host and scheme, and a raw HTTP/1.1 GET request is constructed and converted to bytes.

Before sending, the request is encrypted in layers starting from the exit node's key and working backwards to the entry node's key. The entry node's layer is on the outside; the exit node's layer is the innermost. Only the exit node will ever see the plaintext request:

The wrapped payload is sent as a `DATA_TRANSFER_REQUEST` to the entry node, carrying the circuit ID, target host and port, and the encrypted data.

```
byte[] encrypted = requestBytes;
for (int i = currentHop - 1; i >= 0; i--) {
    encrypted = crypto.encryptAES(encrypted, keys.
        get(i));
}
```

Listing 2: Layered encryption of the outgoing HTTP request

```
java -jar target/zmix-1.0-SNAPSHOT.jar node.
    bootstrap.properties --gui
java -jar target/zmix-1.0-SNAPSHOT.jar node.peer.
    test1.properties
java -jar target/zmix-1.0-SNAPSHOT.jar node.peer.
    test2.properties
java -jar target/zmix-1.0-SNAPSHOT.jar node.peer.
    test3.properties
```

Listing 3: Starting nodes locally

5.2 Relay Forwarding and the Exit Node

Each relay decrypts one layer using its session key and checks whether it has a next hop. If it does, it forwards the message onward. If there is no next hop, the node is the exit node — it has just peeled the last layer and holds the original plaintext HTTP request.

The exit node opens a direct TCP connection to the target. Port 443 uses Java's `SSLSocketFactory` for HTTPS, everything else uses a plain socket. It writes the request, reads the full response, encrypts it with its session key, and sends it back as a `DATA_TRANSFER_RESPONSE`.

5.3 Receiving the Response

The response travels back through the circuit in reverse. Each relay re-encrypts the payload with its own key before passing it to its predecessor. By the time the message reaches the initiator it has been wrapped by every relay on the path.

The initiator decrypts the layers in order — entry first, then middle, then exit — recovering the original HTTP response. The HTML body is extracted, saved to the `responses/` directory as a timestamped file, and the GUI is notified to display it.

6 SETUP & DEPLOYMENT

6.1 Prerequisites

Running ZMIX requires Java 21, Maven 3.8 or later for building, and Docker with Docker Compose for the containerised setup. The project builds a fat JAR using the `maven-shade-plugin`, so no separate dependency installation is needed at runtime.

6.2 Running Locally

After building with `mvn package`, nodes are started by passing a properties file as the first argument. The bootstrap node must be started first, followed by any number of peer nodes in separate terminals:

Once at least three peers are connected and the circuit is established, a URL can be entered in the bootstrap node's terminal or GUI. The `–gui` flag enables the Swing interface; omitting it keeps the node terminal-only.

```
docker-compose up --build
docker attach peer1
```

Listing 4: Starting the network with Docker

6.3 Docker Compose

The included `docker-compose.yml` defines four services, and that are: `bootstrap-node` with port 12137 exposed to the host, and `peer1`, `peer2`, `peer3` all on a shared bridge network inside docker, called `onion-network`. The Dockerfile performs a two-stage build: Maven compiles the fat JAR in the builder stage and Eclipse Temurin 21 runs it in the final stage.

Few environment variables are overriding the properties values (`NODE_PORT`, `NODE_BOOTSTRAP`, and `BOOTSTRAP_HOST`). They allow the same image to serve as either bootstrap or peer depending on the Compose service definition.

6.4 Configuration

The key configuration properties and their defaults are listed below:

- `node.port` – TCP port the node listens on (default: 12137).
- `node.bootstrap` – If true, acts as the seed node and does not connect outbound (default: false).
- `node.connections.max` – Maximum simultaneous peer connections (default: 5).
- `node.connections.min` – Maintenance target for minimum connections (default: 3).
- `bootstrap.host / bootstrap.port` – Address of the seed node to connect to on startup.
- `circuit.length` – Number of hops per circuit (default: 3).
- `connection.retry.attempts` – Max reconnect attempts per peer (default: 10).

7 LIMITATIONS & FUTURE WORK

The current implementation demonstrates the core mechanics of onion routing but has several known limitations worth addressing before any real-world use.

7.1 Current Limitations

- **Single circuit per node.** Each node supports only one outbound circuit at a time. Concurrent requests queue behind it.
- **No circuit teardown.** There is no `CIRCUIT_DESTROY` message type. Relay entries accumulate if the initiator disconnects unexpectedly.
- **Unbounded replay cache.** The message ID history in `MessageHandler` is an `ArrayList` that grows without limit. Long-running nodes will eventually consume significant memory.
- **Simplified key derivation.** SHA-256 applied directly to the ECDH output is a shortcut. HKDF (RFC 5869) with a proper salt and context string is the correct approach for production use.
- **No circuit rotation.** The same circuit is reused for all requests until the node restarts. An observer watching long enough could correlate traffic over time.

7.2 Future Work

- Support multiple concurrent circuits with a circuit pool and request routing logic.
- Implement a `CIRCUIT_DESTROY` message and relay garbage collection on disconnect.
- Replace the SHA-256 KDF with HKDF as specified in RFC 5869.
- Add periodic circuit rotation to limit long-term traffic correlation.
- Replace the unbounded replay list with a time-windowed Bloom filter or bounded LRU cache.

8 CONCLUSION

ZMIX demonstrates that the core mechanics of onion routing can be implemented from scratch in plain Java without any external cryptography or networking frameworks. The system successfully anonymizes HTTP traffic across a self-organizing peer-to-peer network — no single node can determine both the origin and destination of a request, which is the fundamental privacy guarantee onion routing is designed to provide.

The architecture is intentionally modular. The event-driven message bus, the protocol registration pattern, and the clean separation between transport, network management, and circuit logic make each component easy to reason about in isolation. The Docker Compose deployment shows the design works in a real multi-container environment with no code changes beyond environment variable injection.

The implementation is a working proof of concept. The limitations noted in Section 7 — particularly the single-circuit constraint, the simplified KDF, and the lack of circuit rotation — are well understood and addressable. As a foundation for further development, ZMIX covers all the essential building blocks: peer discovery, authenticated key exchange, layered encryption, and end-to-end anonymous HTTP proxying.